Base hebergement_db — service hebergement

Hôtel & Restaurant — Dictionnaire des données V1 (MPD Merise)

Ce document explique **chaque table et chaque attribut** des 7 bases de données : ce qu'il contient, **pourquoi** il existe et **quelle logique** l'utilise. Il correspond au **MPD** (modèle physique) de Merise : les fichiers sources sont `services/<service>/prisma/schema.prisma` et les migrations SQL dans `services/<service>/prisma/migrations/`.

---

0. Les conventions (à lire en premier, elles valent pour toutes les tables)

Convention	Exemple	Pourquoi
Clé primaire `id` en UUID	`78072c82-869c-…`	Généré sans demander à la base, impossible à deviner (on ne peut pas essayer `id=1, 2, 3…` pour lire les données des autres), et unique **entre services** : un `reservationId` créé par hebergement peut voyager dans un événement sans collision.
Nom en camelCase dans le code, snake_case en base	`tarifNuit` ↔ `tarif_nuit` (`@map`)	Convention de TypeScript d'un côté, de PostgreSQL de l'autre. `@@map("chambre")` donne le nom de la table.
Montants en `Int` (entier), en Ariary	`85000`	L'Ariary n'a pas de centimes en pratique, et un entier n'a **jamais d'erreur d'arrondi** (contrairement à `0.1 + 0.2` en virgule flottante).
`createdAt` / `updatedAt`	`@default(now())` / `@updatedAt`	Traçabilité : quand la ligne a été créée, quand elle a changé pour la dernière fois. Prisma remplit `updatedAt` tout seul.
`Timestamptz` pour un instant, **`Date`** pour un jour	`checkInLe` vs `dateArrivee`	Un instant (« 14 h 32 à Tana ») a besoin du fuseau horaire ; une date de séjour (« le 15 octobre ») n'a pas d'heure. Mélanger les deux crée des décalages d'un jour.
`Json` pour les textes traduits (`TexteTraduit`)	`{"fr": "Chambre standard", "mg": "Efitra tsotra"}`	Une seule colonne pour toutes les langues. Ajouter l'anglais = ajouter une clé `en`, sans migration.
« sans FK » + « snapshot »	`clientId` (sans FK) + `clientNom` (snapshot)	**Règle de la SOA** : une base n'a pas de clé étrangère vers la base d'un autre service (ce sont des bases séparées). On garde l'**identifiant** (pour retrouver la donnée) et une **copie figée** des champs utiles (pour afficher sans appeler l'autre service, et pour garder la valeur **au moment de l'action** : si le client change de nom, sa facture garde l'ancien).
`version`	`version Int @default(1)`	**Verrou optimiste** (commande, reservation, tache_menage) : chaque modification fait `version + 1` et exige la version lue (`where: { id, version }`). Si deux personnes modifient en même temps, la seconde modifie 0 ligne et reçoit un conflit (409) au lieu d'écraser la première (« mise à jour perdue »). Pour `chambre`, l'incrément sert à **verrouiller la ligne** pendant le check-in (voir cette table).
Enum	`statut StatutReservation`	Liste fermée de valeurs, vérifiée par la base : une faute de frappe (`"CONFIMEE"`) est impossible.

Convention	Exemple	Pourquoi
Contraintes `CHECK` / `EXCLUDE` dans les migrations	`CHECK (quantite >= 0)`	Prisma ne sait pas les écrire dans `schema.prisma` : elles sont ajoutées **à la main** dans le SQL de migration. C'est la **dernière ligne de défense** : même un bug dans le code ne peut pas les violer.

Les 3 tables techniques (présentes dans plusieurs services)

`outbox_evenement` — patron *Transactional Outbox* (hebergement, facturation, menage, stock, restaurant, comptabilite)

Attribut	Type	Description et logique
`id`	uuid	Sert aussi d'**identifiant de l'événement** (les consommateurs s'en servent pour l'idempotence).
`type`	texte	Nom de l'événement : `reservation.creee`, `menage.termine`… (liste dans `packages/commun/src/evenements.ts`).
`payload`	json	Le contenu de l'événement, validé par un schéma Zod.
`createdAt`	instant	Ordre de publication.
`publieLe`	instant, **nul tant que non publié**	Le relais lit les lignes où `publieLe` est nul, les envoie dans Redis/BullMQ, puis remplit la date. Index `(publieLe, createdAt)` pour trouver vite les événements en attente.

Pourquoi : l'événement est écrit dans **la même transaction** que la donnée métier. Soit les deux sont enregistrés, soit aucun. Si Redis est en panne, l'événement attend ici et part plus tard : il n'est jamais perdu.

`evenement_traite` — consommateur idempotent

Attribut	Type	Description et logique
`evenementId`	uuid, **clé primaire**	L'id de l'événement reçu. Inséré **avant** de traiter, dans la même transaction que l'effet. Si le même événement arrive deux fois, l'insertion viole la clé primaire, la transaction est annulée : **aucun double effet** (jamais deux tâches de ménage pour le même séjour). Utilisé par la fonction `traiterUneFois` (`src/outbox.ts` de chaque service). Exception : stock s'appuie sur la clé unique métier `(referenceId, articleId)` de `mouvement`, une transaction par article.
`traiteLe`	instant	Quand il a été traité.

`compteur_<x>` (`compteur_reservation`, `compteur_facture`, `compteur_commande`)

Attribut	Type	Description et logique
`annee`	entier, **clé primaire**	Un compteur par année : la numérotation repart à 1 chaque 1er janvier.
`dernierNumero`	entier	Dernier numéro donné. Un `upsert` + `increment` **atomique** garantit que deux créations simultanées n'obtiennent jamais le même numéro → `RES-2026-000042`, `FAC-2026-000123`, `CMD-2026-000045`. Pour les factures, la numérotation **sans trou** est une obligation comptable.

2. Base `hebergement_db` — service hebergement (chambres, réservations, séjours)

Table `categorie_chambre` — un type de chambre vendu

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant.
`code`	enum `CodeCategorie`, **unique**	oui	`STANDARD`, `SUITE_SENIOR`, `SUITE_PRESTIGE`. Code stable, utilisable dans les URL et les traductions (le libellé, lui, peut changer).
`libelle`	json (traduit)	oui	Nom affiché : `{"fr": "Suite senior", "mg": "…"}`.
`description`	json (traduit)	non	Texte de présentation.
`capacite`	entier, **CHECK > 0**	oui	Nombre maximal de personnes.
`tarifNuit`	entier (Ar), **CHECK > 0**	oui	Prix **par défaut** d'une nuit, utilisé hors saison.

Le client réserve une catégorie, pas une chambre : c'est l'hôtel qui choisit la chambre libre (la plus simple à gérer pour lui).

Table `chambre` — une chambre physique

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant.
`numero`	texte, **unique**	oui	« 204 ». Texte et non nombre : certains hôtels ont « 12B ».
`etage`	entier	oui	Pour le tableau des chambres de la réception.
`categorieId`	uuid, **FK → categorie_chambre** (suppression interdite)	oui	Type de la chambre. On ne peut pas supprimer une catégorie qui a encore des chambres. Index pour trouver vite les chambres d'une catégorie.
`etatMenage`	enum `EtatMenage`	oui (défaut `PROPRE`)	`PROPRE` → `SALE` (au check-out) → `EN_NETTOYAGE` (ménage démarré) → `PROPRE` (événement `menage.termine`). **Le check-in exige `PROPRE`.**
`horsService`	booléen	oui (défaut faux)	Chambre en travaux : jamais attribuée à une réservation.
`version`	entier	oui	**Écrire pour verrouiller** : au check-in, `updateMany({ where: { id, etatMenage: PROPRE, horsService: false }, data: { version: +1 } })` vérifie que la chambre est propre **et** verrouille sa ligne jusqu'au commit, en une seule instruction. Personne ne peut la passer à `SALE` entre la vérification et l'installation du client. (Deux réservations de la même chambre aux mêmes dates, elles, sont empêchées par la contrainte `EXCLUDE` de `reservation`.)

Tables `saison` et `tarif_saison` — prix différents selon la période

`saison`

Attribut	Type	Description et logique
`id`	uuid	Identifiant.

Attribut	Type	Description et logique
`nom`	json (traduit)	« Haute saison », « Fêtes de fin d'année ».
`dateDebut`, `dateFin`	date	Période **incluse** (`[]`). Contrainte `EXCLUDE saison_sans_chevauchement` : deux saisons ne peuvent pas se chevaucher, sinon une nuit aurait deux prix.

`tarif_saison` — table **d'association** (relation plusieurs-à-plusieurs entre saison et catégorie, avec un attribut porté)

Attribut	Type	Description et logique
`saisonId`	uuid, FK → saison (cascade)	
`categorieId`	uuid, FK → categorie_chambre	
`tarifNuit`	entier (Ar)	Le prix d'une nuit de **cette** catégorie pendant **cette** saison.
Clé primaire	`(saisonId, categorieId)`	Clé **composée** : un seul prix par couple saison × catégorie. C'est exactement l'**association porteuse de données** du MCD Merise.

Table `reservation` — une réservation, puis un séjour

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant technique.
`reference`	texte, **unique**	oui	`RES-2026-000042` : lisible, à donner au téléphone (vient de `compteur_reservation`).
`chambreId`	uuid, FK → chambre	oui	La chambre attribuée par l'hôtel (peut changer au check-in si elle n'est pas propre).
`clientId`	uuid (auth, **sans FK**)	**non**	Le compte du client s'il a réservé en ligne ; **nul** pour une réservation au comptoir d'un client sans compte. Index pour « Mes réservations ».
`clientNom`, `clientTelephone`, `clientEmail`	texte (**snapshots**)	oui	Coordonnées **au moment de la réservation** ; présentes même sans compte. L'email est obligatoire : le reçu part par email.
`pieceIdentiteType`, `pieceIdentiteNumero`	texte	oui	CIN ou passeport + numéro : exigé par la réglementation hôtelière, vérifié au check-in. **Donnée personnelle sensible.**
`dateArrivee`, `dateDepart`	date	oui	Le séjour. **CHECK** `date_depart > date_arrivee` (au moins une nuit).
`montantTotal`	entier (Ar), CHECK ≥ 0	oui	**Calculé par le serveur** (somme des nuitées), jamais envoyé par le navigateur.
`statut`	enum `StatutReservation`	oui	Machine à états : `EN_ATTENTE_PAIEMENT` → `CONFIRMEE` (paiement reçu) → `EN_COURS` (check-in) → `TERMINEE` (check-out) ; ou `ANNULEE` (non payée à temps, ou par la réception). Index pour filtrer par statut.

Attribut	Type	Obligatoire	Description et logique
`canal`	enum `CanalReservation`	oui	`SITE_WEB`, `RECEPTION`, `FACEBOOK`, `ASSISTANT_IA` (prévu V2). Sert aux statistiques « nuitées par canal » de la comptabilité.
`langue`	enum	oui	Langue des messages envoyés pour cette réservation.
`expireLe`	instant	non	Fin des **30 minutes** pour payer. Un job BullMQ différé l'annule à cette heure s'il est toujours `EN_ATTENTE_PAIEMENT`. Remis à nul une fois confirmée ou annulée.
`checkInLe`, `checkOutLe`	instant	non	Heures réelles d'arrivée et de départ.
`version`	entier	oui	Verrou optimiste : par exemple, l'expiration et la confirmation de paiement arrivent en même temps ; une seule gagne.

La règle la plus importante de la base — contrainte `EXCLUDE` `reservation_sans_chevauchement` :

```
EXCLUDE USING gist ("chambre_id" WITH =, daterange("date_arrivee", "date_depart", '[)') WITH &&)
WHERE ("statut" IN ('EN_ATTENTE_PAIEMENT', 'CONFIRMEE', 'EN_COURS'))
```

Traduction : « deux réservations **actives** de la **même chambre** (`=`) ne peuvent pas avoir des périodes qui **se chevauchent** (`&&`) ». `'[)'` exclut la date de départ : un départ le 13 et une arrivée le 13 sont permis. Les réservations annulées ou terminées ne comptent pas. C'est **PostgreSQL** qui empêche le surbooking, même avec deux clics à la même milliseconde. (`btree_gist` est l'extension qui permet de combiner `=` et `&&` dans une même contrainte.)

Index `(chambreId, dateArrivee)` : pour chercher vite une chambre libre.

Table `reservation_nuitee` — le détail nuit par nuit

Attribut	Type	Description et logique
`reservationId`	uuid, FK → reservation (cascade)	
`nuit`	date	Une ligne par nuit du séjour.
`tarifApplique`	entier (Ar)	Le prix **réellement appliqué** cette nuit-là (tarif de saison ou tarif par défaut), **figé** : si l'admin change les tarifs demain, la réservation ne change pas.
`saisonId`	uuid, FK → saison (mise à nul si la saison est supprimée)	Quelle saison a fixé ce prix (nul = tarif par défaut).
Clé primaire	`(reservationId, nuit)`	Une seule ligne par nuit.

`montantTotal` de la réservation = somme des `tarifApplique`. C'est cette table qui servira à afficher le « détail des nuitées » (F5 du backlog).

Table `cle_idempotence` — rejouer une requête sans créer de doublon

Attribut	Type	Description et logique
`cle`	uuid, **clé primaire**	La valeur de l'en-tête `Idempotency-Key` envoyé par le front.

Attribut	Type	Description et logique
`route`	texte	Sur quelle route elle a servi.
`utilisateurId`	uuid	L'auteur : sinon, quelqu'un qui devinerait la clé pourrait relire la réponse d'un autre client.
`reponse`	json	La réponse déjà renvoyée.
`createdAt`	instant	Purge après 7 jours.

Logique : le client clique « Réserver », le réseau coupe, le navigateur renvoie la requête avec la **même clé** → la base renvoie la réponse déjà enregistrée au lieu de créer une deuxième réservation.

---

9. Ce qui changera avec le SaaS (Phase 2)

- **Toutes** les tables métier recevront `hotelId` (sauf les tables propres au client global).
- Les uniques deviendront **par hôtel** : `(hotelId, numero)` pour les chambres, `(hotelId, code)` pour les catégories et les articles, compteurs `(hotelId, annee)`.
- `utilisateur` deviendra **global** ; une nouvelle table `membre (utilisateurId, hotelId, role)` portera les rôles du personnel.
- `outbox_evenement` et les événements porteront le `hotelId`.
- Les politiques **RLS** de PostgreSQL filtreront par `hotelId` (personnel) ou par `clientId` (client).